

指挥官战术对抗

核心代码片段注释 · 架构导读

本文从仓库中抽出最能代表系统骨架的代码：命令协议、固定时间步模拟、战争迷雾、流场寻路、中文规则解析、双层 AI、语音识别管线。注释侧重“为什么这样写”，便于答辩讲解与二次开发。

1. 公共协议：命令与观察快照 (protocols.py)

所有输入（鼠标/文字/语音/AI）都必须变成 CommandEnvelopeV1；AI 与界面只能读 ObservationSnapshotV1。Pydantic + extra=forbid 保证协议不可被随意塞字段。

```
class CommandEnvelopeV1(ProtocolModel):
    """一次结构化军令：谁、何时、从哪来、干什么。"""
    command_id: str
    faction: Faction
    issued_tick: int # 发出时模拟帧，用于过期检查
    source: CommandSource # mouse/text/voice/local_ai/...
    payload: CommandPayloadV1 # 带 discriminator 的联合类型

class ObservationSnapshotV1(ProtocolModel):
    """某阵营“依法能看见什么”。不含未发现敌军坐标。"""
    faction: Faction
    tick: int
    own_units: tuple[UnitObservationV1, ...]
    visible_enemies: tuple[UnitObservationV1, ...]
    known_terrain: bytes # 仅已探索地形
    known_facilities: tuple[dict, ...]
```

要点：公平靠数据边界。AI 改代码也拿不到迷雾外信息，因为 snapshot 构造时就过滤了。

2. 命令入口：execute 分发与过期 (world.py)

World.execute 是唯一权威入口。先检查是否过期，再按 payload.kind 分发；只有 ACCEPTED 才写入 command_log，便于回放复现。

```
def execute(self, command: CommandEnvelopeV1) -> CommandResultV1:
    # 网络/异步返回过晚时直接作废，避免“幽灵指令”
    if self.tick - command.issued_tick > self.balance.world.simulation_hz * 10:
        return self._result(command, CommandStatus.EXPIRED, "command_expired", "...")
    payload = command.payload
    if isinstance(payload, MovePayloadV1):
        result = self._execute_move(command, payload)
    elif isinstance(payload, BuildPayloadV1):
        result = self._execute_build(command, payload)
    # ... 其余 kind 同理
    if result.status == CommandStatus.ACCEPTED:
        self.command_log.append(command) # 回放只记结构化命令
    return result
```

要点：输入层不能直接改单位；回放不重新调用语音/LLM，只重放 command_log。

3. 固定时间步主循环：step (world.py)

一帧里按固定顺序推进：计时器 → 移动 → 碰撞 → 桥面约束 → 战斗 → 工程 → 胜负 → 感知。碰撞、战斗、感知做了分频，降低千人规模开销。

```
def step(self, steps: int = 1) -> None:
    for _ in range(steps):
        if self.outcome != GameOutcome.ONGOING:
            return
        self._update_timers() # 冷却 / 战术状态 / 耐力恢复
        self._move_units() # 沿流场与目标推进
        if self.tick % 5 == 1:
            self._resolve_collisions() # 空间哈希，不必每帧
            self._constrain_units_to_bridge_decks()
            self._restore_illegal_river_entries()
        if self.tick % 4 == 0:
            self._resolve_combat()
            self._update_engineering()
            self._check_victory()
        if self.tick % 10 == 0:
            self._perception.update(self) # 迷雾与敌军可见性
        self.tick += 1
```

要点：渲染与模拟解耦；同一帧内统一结算伤害，减少更新顺序造成的不公平。

4. 永久探索战争迷雾 (perception/fog.py)

explored 一旦揭开就永久保留；视野半径按地形衰减。敌军历史情报写入 history，供 UI 显示“最后出现位置”。

```
class FogOfWar:
    def __init__(self, rows, cols, tile_size):
        self.explored = np.zeros((2, rows, cols), dtype=np.bool_) # 两阵营各一份

    def update(self, world: World) -> None:
        for faction in (Faction.PLAYER, Faction.ENEMY):
            active = world.units.active(faction)
            cols = (world.units.x[active] / world.subpixels // self.tile_size)
            rows = (world.units.y[active] / world.subpixels // self.tile_size)
            radii = np.ceil(world.units.vision[active] * vision_table[terrain] / tile)
            for row, col, radius in np.unique(np.column_stack((rows, cols, radii)), axis=0):
                self._reveal_cells(int(faction), int(row), int(col), int(radius))
            self.explored[int(faction)] |= self.visible[int(faction)]
# 只把“当前可见敌军”写入 history —— 离开探索区即消失
```

要点：逻辑判定与云雾动画分离；未探索区右键不得泄露真实地形。

5. 共享流场寻路 (simulation/navigation.py)

千人不各自 A*：按目标格缓存整张 FlowField，单位只查局部方向。桥梁用定向矩形 mask 打开河格，缓存随地形 revision 失效。

```
class FlowFieldCache:
    def get(self, target_x, target_y, movement="land") -> FlowField:
        key = (col, row, movement, self.map.revision)
        field = self.cache.get(key)
        if field is None:
            field = self._build(col, row, movement) # Dijkstra 积分场
            self.cache[key] = field
        if len(self.cache) > 48: # 简单 LRU 上限
            self.cache.pop(next(iter(self.cache)))
        return field

    def set_bridges(self, bridges, radius):
        """仅已完成桥梁覆盖的河格对陆军可通行。"""
        # 横/竖桥用矩形判定，避免“点圆”误开河道
        mask |= ((centers_x - bx) <= length/2) & ((centers_y - by) <= HALF_WIDTHH)
```

要点：O(单位数) 查表代替 O(单位数 × A*)；每个阵营寻路缓存只含自己已知的桥。

6. 中文规则命令解析 (commands/parser.py)

离线主路径：清洗标点 → 解析选择集 → 解析目标 → 生成 Envelope。只使用 Observation 里已知设施/村庄，天然防“看见不存在的敌军”。

```
def parse(self, text, observation) -> CommandResultV1:
    cleaned = re.sub(r"[，。！？，!?!]", "", text.strip())
    selection = self._selection(cleaned, observation) # “第1组” / “工兵” / “20名步兵”
    target = self._target(cleaned, observation) # “北部中央” / 村庄名/坐标词
    if "摧毁" in cleaned or "攻击设施" in cleaned:
        targets = [f for f in observation.known_facilities
                    if f["faction"] != observation.faction] # 只打已知敌方设施
    envelope = CommandEnvelopeV1(..., payload=AttackFacilityPayloadV1(...))
# 解析失败返回 REJECTED + 中文原因，供指挥面板展示
```

要点：离线也能完整游玩；LLM 只是同一校验链路上的可选前端。

7. 双层本地 AI (ai/local.py)

战略层低频定目标，战术层高频处理接敌；decide 只吃 ObservationSnapshot。难度表控制间隔、攻击性、撤退阈值与是否允许鼓舞。

```
_DIFFICULTY_PROFILES = {
"normal": {"strategy_interval_ticks": 40, "tactical_interval_ticks": 5,
"aggression": 0.7, "retreat_hp_ratio": 0.25, ...},
}

class LocalStrategicAI:
    """A fair controller that only accepts a filtered observation."""
    def decide(self, observation) -> list[CommandEnvelopeV1]:
        commands = []
        if self._tactical_ready(observation.tick):
            commands.extend(self._tactical_decisions(observation))
        if self._strategy_ready(observation.tick):
            commands.extend(self._strategic_decisions(observation))
        return commands[: int(self.config["max_simultaneous_tasks"])]
```

要点：AI 输出的也是 CommandEnvelope，与玩家同一执行路径，便于审计是否作弊。

8. 语音识别管线 (input/voice.py + app/game.py)

按住 V 采音 → 过短/静音直接丢弃 → 优先在线 (DeepSeek/专用 ASR) → 失败回落本地 Whisper → 文本再走规则/LLM 军令。

```
# OnlineSpeechRecognizer: SPEECH_API_KEY 优先，否则复用 DEEPSEEK_API_KEY
# 请求 {base}/audio/transcriptions (OpenAI 兼容 multipart)

def _transcribe_voice_samples(self, samples: bytes) -> str:
    if self.online_enabled and self.speech.online.available:
        try:
            text = self.speech.online.transcribe(samples, ..., require_audible=False)
            if text:
                return text
        except VoiceUnavailable as exc:
            logger.warning("online speech failed, falling back to local: %s", exc)
    return self.whisper.transcribe(samples, ..., require_audible=False)

# 结果写入 command_input 后 _submit_text():
# 规则解析 → 失败且在线可用时 DeepSeek → 仍失败给中文原因
```

要点：语音不是黑盒按钮；每一步有失败原因；离线文字军令始终可用。

附：推荐阅读顺序

protocols.py → simulation/world.py (execute/step) → perception/fog.py → simulation/navigation.py → commands/parser.py → ai/local.py → input/voice.py。跑测试可用：pytest tests/unit/test_commands.py tests/unit/test_fog.py tests/unit/test_voice.py

本 PDF 为教学/答辩导读，片段为节选并加了注释，完整实现以仓库源码为准。